DESeq2 Tutorial

Dr. Rachel Howard

Contents

DESeq2: Setup and Requisites (`tximport`) **2**

Alignment . . . 2

Alignment-free (aka pseudoalignment/quasi-mapping) . . . 3

Gene-level vs Transcript-level (Isoform-level) Counts . . . 4

Visual Analogy: A Gene as a Tree . . . 6

Using `tximport` with Salmon data . . . 6

Running DESeq2 **11**

1. Prepping metadata . . . 11

2. Build a DESeqDataSet (DDS) . . . 12

3. Run the DESeq pipeline . . . 14

Understanding DESeq2: Results Tables . . . 14

Understanding DESeq2: Dispersion . . . 15

Visualization of DEGs: Volcano Plots . . . 17

Congratulations! . . . 20

This is an R Markdown Notebook where we will go over the DESeq2 package for DE analysis. When you execute code within the notebook, the results appear beneath the code.

First, let's get the DESeq2 package installed and loaded. If you haven't installed DESeq2 before, you will need to install it from Bioconductor using the BiocManager package. We're also going to load in some practice data!

The data used in this workflow is stored in the airway package that summarizes an RNA-seq experiment wherein airway muscle cells were treated with an anti-inflammatory steroid. In the experiment, four primary human airway muscle cell lines were treated with 1 micromolar dexamethasone for 18 hours. For each of the four cell lines, there is a treated and an untreated sample. See more info on NCBI PubMed.

DESeq2: Setup and Requisites (`tximport`)

Let's get our packages and data loaded in. You need to load your libraries in *every time* you open R.

```
if (!require("BiocManager", quietly = TRUE))
    install.packages("BiocManager") ## run if you need to install BiocManager

BiocManager::install("DESeq2") # run if you need to install DESeq2
BiocManager::install("airway") # run if you need to install the airway dataset
BiocManager::install("tximport") # run if you need to install the tximport package
BiocManager::install("tximportData")
BiocManager::install("readr") # for reading sample metadata

## Always load your libraries!
library(DESeq2)
library(ggplot2)
library(airway)
library(tximport)
library(tximportData)  # contains example tx2gene mappings
library(readr)
library(dplyr)

## Let's load in the airway data that we installed.
dir <- system.file("extdata", package="airway", mustWork=TRUE)
list.files(dir)

## We want the output files specifically from Salmon, which we used to count our RNA-seq reads

list.files(file.path(dir, "quants"))
```

Let's talk about input data. DESeq2 uses a ***gene-level count matrix*** for differential expression testing. It usually has sample names as the columns, and genes as the row names. In the cells are the # reads that gene had in that sample (# reads per gene × sample).

There are two different approaches to get to that point:

1. Alignment, then counting
2. Alignment-free (quasi-mapping, pseudoalignment)

Let's go over the details of both.

Alignment

When we align with tools like Bowtie2 or STAR, we are aligning our transcript reads to either a reference genome/transcriptome, or we are aligning the reads back to a transcriptome we made with ***de novo*** assembly (Trinity). This is a **full** alignment of each read against your transcriptome.

After alignment, then we can use tools like RSEM to quantify our genes.

Your pipeline ends up looking something like this:

FASTQ → Align to genome (Bowtie2/STAR/HISAT2) → Count reads (RSEM/featureCounts/HTSeq) → Gene count matrix → DESeq2

There are usually two major outputs (per sample) from this workflow, especially when you're using RSEM:

1. **Gene count files** (genes.results) that can go directly into DESeq2 because counts are gene level estimates.
2. **Isoform count files** (isoforms.results) that *cannot* go directly into DESeq2 because counts are transcript-level estimates.

Pros

- This is the trusted and traditional approach in 'omics (widely accepted)
- You get very rich information that is useful for investigating:
  - alternative splicing
  - SNPs/variants
  - structural features of transcripts (exon/intron, isoforms, etc)
- The outputs (BAMs) can be used in a lot of other downstream pipelines (variant calling, IGV, etc).

Cons

- Slow and computationally expensive (alignments take hours → days)
- Storage heavy (BAM files are usually very large, 10-100GB/sample)
- Can be overkill if you just want counts

Alignment-free (aka pseudoalignment/quasi-mapping)

If you choose to use alignment-free quantification tools (like Salmon), there is usually a two step process before getting your counts into DESeq2:

1. Quantification of transcripts Tools like Salmon and Kallisto do **NOT** align/map reads. Salmon skips true, base-by-base alignment and builds an index of your transcriptome, assigning reads to compatible transcripts via quasi-mapping.

- It just figures out which transcripts each read is compatible with.

Ultimately, quasi-mapping tools are much faster, but because they are not aligning base-to-base, the output has lower resolution. The output is an **estimated counts per transcript** files (.sf) , with bias-corrected effective lengths.

- This is comparable to the isoforms.results files you get from Bowtie2 + RSEM, so it **cannot** go directly into DESeq2.

2. Aggregate to gene-level counts Using a tool called `tximport`, we can collapse transcript-level estimates to gene-level counts to get the appropriate matrix.

It is important to note that Salmon is alignment-free by default, but alignment-friendly if you want. You can input BAM files into Salmon for finer resolution, but pseudoalignment is what it's built for.

You're pipeline ends up looking something like this:

FASTQ → Quantify transcripts (Salmon/Kallisto) → tximport → Gene count matrix + offsets → DESeq2

Key difference: RSEM vs Salmon/Kallisto

- **RSEM** (and other tools): produces both **gene-level** and **transcript-level** outputs. You can use the `genes.results` directly in DESeq2.
- **Salmon/Kallisto** (and other tools): produce **transcript-level outputs only**. You must use `tximport` (or similar) to aggregate up to genes before using DESeq2.

All of this to say, we need to have the right data, formatted in the correct way, in order to perform DESeq2.

Let's break this down a little more...

Gene-level vs Transcript-level (Isoform-level) Counts

Understanding the distinction between gene-level and transcript/isoform-level counts is key for differential expression analysis.

---

1. Gene-level counts

- **Definition**: The total number of sequencing reads assigned to a *gene locus*, regardless of which transcript isoform those reads came from.
- **Example**: The human gene **TP53** has multiple isoforms. Gene-level counts just ask: *"How many reads map to TP53 in total?"*
- **Why it matters**:
  - Robust and less ambiguous

  - Standard input for **DESeq2**

- **Produced by**: `featureCounts`, `HTSeq`, `RSEM`, or by collapsing transcript-level counts with `tximport` (our method of choice).

---

2. Transcript-level counts

- **Definition**: Reads assigned to a *specific transcript* (a unique mRNA isoform of the gene).
- **Example**: For TP53, you may see:
  - **TP53-201** (ENST00000269305)
  - **TP53-202** (ENST00000413465)
- **Why it matters**:
  - Useful for studying *alternative splicing* or *isoform switching*.
  - More uncertain because many reads are compatible with multiple transcripts.
- **Produced by**: Salmon, Kallisto
- **Note**: DESeq2 does **not** directly use transcript-level counts; instead, these are aggregated with `tximport`.

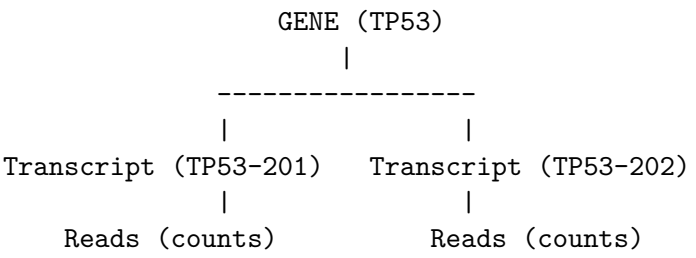
---

3. Isoform-level counts

- **Definition**: Essentially the same as transcript-level counts — an *isoform* is just a transcript variant of a gene.
- **Terminology**:
  - Bioinformatics often says **transcript-level**
  - Wet lab/molecular biology often says **isoform-level**

Visual Analogy: A Gene as a Tree

Think of a gene like a tree:



Level	Example	What is counted?
Gene-level	TP53	All reads mapping to **any isoform** of TP53
Transcript-level	TP53-201, TP53-202	Reads assigned to **specific transcript** isoforms
Isoform-level	(same as transcript-level)	Just another way of saying transcript-level

- **Gene-level counts**: Add up all reads across all branches = total for the *whole tree*.
- **Transcript/Isoform-level counts**: Count reads on each branch separately.

Why this matters for DESeq2

- DESeq2 is designed for **integer, gene-level counts**.
- Transcript/isoform counts are estimated and often fractional.
- **Solution**: Use `tximport` to sum/scale transcript counts into **gene-level counts** before running DESeq2.

In the airways data set, we have access to the data directly from Salmon, which means we need to aggregate to a **gene-level** count matrix before using DESeq2.

Using `tximport` with Salmon data

The `tximport` package allows us to take **transcript-level abundance estimates** (from Salmon, Kallisto, or RSEM isoforms) and summarize them to **gene-level counts** suitable for DESeq2.

We already installed and loaded in the `tximport` and `readr` libraries, so let's jump right in!

Step 1: Prepare the file paths The first thing we are going to do is prepare the file pathways to our sample directories. In each of our sample directories, we will have a `quant.sf` file.

We first create a vector of file paths, and name them according to sample IDs.

```
# Rerun to see info about our quants directory again.
list.files(file.path(dir, "quants"))
# "SRR1039508" "SRR1039509"

# Let's list and vector out the sample folders inside quants
# In our airways data set, we have two samples in our quants directory. We are going create a
sample_dirs <- list.files(file.path(dir, "quants"), full.names = TRUE)

# Append "quant.sf.gz" to each directory path
files <- file.path(sample_dirs, "quant.sf.gz")

# Name the vector with sample IDs (extracted from folder names)
names(files) <- basename(sample_dirs)

files
```

So, in the above code, we have specified exactly where to find the Salmon count matrix outputs for both of our samples by building a pathway:

```
"/Library/Frameworks/R.framework/Versions/4.5-x86_64/Resources/library/airway/extdata/quants/SI
```

Let's run this little chunk of code to make sure each file really exists :) It should return true!

```
all(file.exists(files))
```

Step 2: Transcript → Gene mapping `tximport` needs to know which transcripts belong to which genes.

You can get this mapping from Trinity (Trinity.fasta.gene_trans_map), your annotation (GTF/GFF or Ensembl), or other methods. Usually this is a simple 2-column data frame with transcript IDs and corresponding gene IDs.

In the `airways` data set, we have a `GTF` file already made for us to practice with called `Homo_sapiens.GRCh37.75_subset.gtf` (`hg19`). Let's dive in some more:

What is `hg19`?

- **hg19** = Human Genome version 19 (GRCh37), released by the Genome Reference Consortium in 2009.

In the Airways dataset

- The **airway** Bioconductor dataset is based on human airway smooth muscle cells.
- The reference annotation file used comes from **hg19**, or the `Homo_sapiens.GRCh37.75_subset.gtf` file in the data set.
- That's why you saw `hg19` appear in the tutorial.

We have to be careful here, because a GTF is a tab-delimited sheet, usually with about 9 columns. Let's take a peak at it.

```
# Read the GTF correctly (tab-delimited, not CSV)
gtf <- read_tsv(
  file.path(dir, "Homo_sapiens.GRCh37.75_subset.gtf"),
  comment = "#",   # skip metadata lines starting with "#" (this is usually just data from ENS
  col_names = TRUE
)

# GTF has 9 required columns. Let's name them properly:
colnames(gtf) <- c(
  "seqname", "source", "feature", "start", "end", "score",
  "strand", "frame", "attribute"
)
```

So when we take a look at the gtf data frame, we see a bunch of different data from the `hg19` GTF. However, because we are just using this to map transcript IDs back to gene IDs, we **ONLY** care about the "attribute" column, which has both the `transcript_id` and the `gene_id` for a given RNA sequence (row). A cell in the "attributes" column will typically have information like:

```
"gene_id \"ENSG00000175262\"; transcript_id \"ENST00000468348\"; gene_name \"C1orf127\"; gene_s
```

Again, we are only using this to map transcripts back to genes, so we only want the `gene_ID` and the `transcript_ID` information.

Let's use piping (`dplyr`) to manipulate `gtf` into the 2-column data frame with transcript IDs and corresponding gene IDs per sequence.

```
# Extract transcript-to-gene mapping information (tx2gene)
# GTF "attribute" column has semi-colon separated key-value pairs

tx2gene <- gtf %>%

  # Keep only rows where the GTF feature type is "transcript"
  # (those are our actual mRNA seqs)

  filter(feature == "transcript") %>%
```

```r
  # Create new columns (gene_id and transcript_id) by extracting values
  # from the "attribute" column using regular expressions.

  mutate(
    # Extract the gene_id from text like: gene_id "ENSG00000175262";
    gene_id = sub('.*gene_id "([^"]+)".*', '\\1', attribute),

    # Extract the transcript_id from text like: transcript_id "ENST00000468348";
    transcript_id = sub('.*transcript_id "([^"]+)".*', '\\1', attribute)
  ) %>%

  # Keep only the two columns needed for tximport:
  # gene_id (parent gene identifier)
  # transcript_id (unique transcript identifier)
  select(transcript_id, gene_id)

# Check out the df!
head(tx2gene)
```

Let's walk through what we just did:

- `read_csv()` assumes commas, but GTFs are tab-delimited, so we needed to use `read_tsv()` instead.
- `comment = "#"` skips the header lines like `#!genome-build ....`
- We set column names manually since GTFs don't come with headers.
- The `attribute` column holds key–value metadata (like gene_id, transcript_id). We extract those with regex.
- Finally, we build a two-column `tx2gene` mapping frame (the format that `tximport` expects).

You should see a two-column data frame with transcript IDs and the corresponding gene IDs:

transcript_id	gene_id
ENST00000335137	ENSG00000186092
ENST00000448914	ENSG00000284733
...	...

In our lab, working directly from coral genomic data, you would grab the transcript names directly from a genome that has already been assembled. This is the process Daniela has used in the past for some of her analyses:

Real, coral example from Daniela: *Tximport - Count Matrix Generation*

First we read in our raw counts from Salmon and turn them into count matrices using Tximport.

In the HPC, I extracted the descriptor lines in the Past genome fasta file to obtain transcript IDs.

```
#SERVER= hpcr8o2rnp@uta.edu
#DIR = /home/dxg2725/Projects/past_comparison/genome/

#grep ">" Pastreoides_transcripts_v1.fasta > Past_transcript_names.txt
```

In Excel, I cleaned the "Past_transcript_names.txt" files to remove all the extra information and to leave behind only the transcript IDs.

I then created the "Past_tx2gene.csv" file by doubling the transcript ID column. I am doing this since I am starting without annotating.

```
# past_tx2gene <- read.csv("./DESeq/Past_tx2gene.csv", header = FALSE)
# head(past_tx2gene)
#
# #Generate recursive path to quants directories
# past_samples <- read.table("./DESeq/past_sample_list.csv", header = TRUE)
# past_samples
#
# setwd("./Salmon/quants_directories/host/")
# past_files <- file.path(past_samples$sample, "quant.sf")
# names(past_files) <- paste0("sample", 1:19)
# all(file.exists(past_files))
# # Should return '[1] TRUE'
```

Let's continue on with the `airways` data set.

Step 3: Run `tximport` Now we are good to actually run `tximport` with our `tx2gene` data frame we made! Again, we are aggregating the Salmon transcript-level quantification to gene level counts.

```
# Run tximport
txi <- tximport(
  files,              # a named character vector of paths to the Salmon quant.sf files
  type = "salmon",    # tells tximport what type of quantification files you're providing
  tx2gene = tx2gene, # the 2-column dataframe mapping transcript IDs -> gene IDs
  ignoreTxVersion = TRUE # tells tximport to strip off version numbers from transcript IDs
                         # ex "ENST00000456328.2" becomes "ENST00000456328"
                         # This helps match IDs between quant.sf and tx2gene
)

# Inspect the output
```

```
names(txi) # contents of the txi object
dim(txi$counts)
head(txi$counts)
```

Okay so what does our new `tximport` object `txi` actually contain?

- `txi$counts` → gene-level counts
- `txi$abundance` → TPM values
- `txi$length` → effective transcript lengths

You've successfully run `tximport`! This is usually the first step for making your Salmon quantification files usable for `DESeq2`!

For our purposes, we will not be using our `txi` object with DESeq2. DESeq2 will not work on two samples, because it will not treat samples as replicates, and it cannot properly estimate dispersion.

Because of this, we will say goodbye to our `txi` object for now, but knowing how to work `tximport` is **extremely useful** to getting to where we need to go from Salmon into DESeq2.

Great job everyone! Keep going!

Running DESeq2

1. Prepping metadata

Instead of loading the airway dataset as individual files with pathways like we did for `tximport`, we'll load it as a **RangedSummarizedExperiment**. This gives us a ready-to-use Bioconductor object that already includes the RNA-seq counts, sample metadata, and gene ranges—making it much easier to start analyzing the data in DESeq2 without having to manually import and match files. Here is how we do it:

```
data("airway")
```

The first thing we are going to do is make sure our metadata is loaded in and correctly formatted. We can inspect the metadata in our object by pulling up the `colData` slot, which is the metadata for the samples.

```
colData(airway)
```

You should see a dataframe with all of the sample metadata.

The point of the airways experiment was to characterize gene expression and functionality of airway muscle cells in response to an anti-inflammatory steroid called **dexamethasone**. In the metadata we see above, there is a column called **dex**, short for dexamethasone. In the cells of that column, we can see *trt* and *untrt*, shorthand for *treated* and *untreated* respectfully.

This will be our ***design formula*** in our DESeq2 testing. Ultimately, it's a statistical model formula in R that tells DESeq2 how to explain variation in gene counts (differential expression) using our metadata. Since we will be using **dex** in our formula, what we are really asking is: > *For each gene, model its expression as a function of the sample's treatment type (trt or untrt).*

With this design, DESeq2 will fit a **generalized linear model (negative binomial)** for each gene, using the `dex` column (*trt* vs *untrt*) as the explanatory variable. This allows DESeq2 to estimate log2 fold changes and test for significant differences between the two treatment types.

Our metadata looks something like this when we consider the variables we want to be testing:

sample	dex
S1	trt
S2	untrt

What will be happening behind the scenes with DESeq2:

- Compare average counts in *trt* vs *untrt* for each gene,
- Estimate log2 fold changes in expression (by looking at the counts),
- Test whether the difference is statistically significant.

Let's go ahead and give it a go!

2. Build a DESeqDataSet (DDS)

Let's use the **gene-level counts** in the airways dataset to build our DDS object

```
dds <- DESeqDataSet(airway, design = ~ dex)

## Let's look:
dds
```

We can see that:

1. `assay(airway)` contains raw counts.
2. `colData(airway)` contains the sample info (dex = treatment).
3. `design = ~ dex` tells DESeq2 to model gene expression as a function of the treatment.

Sometimes, its helpful to filter out genes that had an average expression (across all samples) of less than 10. This number can vary based on your own experiment and data, but 10 seems to be pretty standard in RNA-seq. Filtering can make your analyses faster, lighter, and easier to interpret in the long run:

- **Saves memory**: fewer rows in the dataset.

- **Speeds up calculations**: DESeq2 doesn't waste time on genes with almost no reads.
- **Improves plots**: low-count/noisy genes don't clutter dispersion plots or MA-plots.

Let's filter our data now. We are filtering based on the **average raw count across all samples**, and only keeping genes with a **mean count of 10 or higher**.

```
keep <- rowMeans(counts(dds)) >= 10
dds <- dds[keep, ]

counts(dds) # inspect the count matrix
```

Here's what we did step-by-step:

1. `counts(dds)`
   - Extracts the **raw counts matrix** from the `DESeqDataSet`.
   - Rows = genes, columns = samples.
2. `rowMeans(counts(dds))`
   - Calculates the **mean raw count** for each gene across all samples.
   - Returns a numeric vector (length = number of genes).
3. `rowMeans(counts(dds)) >= 10`
   - Compares each gene's mean to `10`.
   - Produces a logical vector (`TRUE`/`FALSE`) where `TRUE` means the gene's mean raw count is at least 10.
   - Stored in the variable `keep`.
4. `dds <- dds[keep, ]`
   - Uses the logical vector to subset the dataset by rows.
   - Keeps only genes where `keep` is `TRUE`.
   - All samples (columns) remain.

A lot of this code happens behind the scenes, so it can be hard to imagine and think about what this all actually looks like. Let's break it down.

Worked example Suppose we have a small counts matrix:

Gene	S1	S2	S3
GeneA	0	5	15
GeneB	12	8	10
GeneC	100	120	110

- **GeneA**: mean = (0 + 5 + 15) / 3 = 6.7 → **discarded**
- **GeneB**: mean = (12 + 8 + 10) / 3 = 10 → **kept**
- **GeneC**: mean = (100 + 120 + 110) / 3 = 110 → **kept**

Result: only GeneB and GeneC are retained in the filtered dataset.

3. Run the DESeq pipeline

Now let's run DESeq on the actual data object we just made (`dds`):

```
test <- DESeq(dds)

## Make a results table from the DESeq object!
res <- results(test)
head(res)
```

Above, you should see the first five lines of your results table from DESeq2.

Understanding DESeq2: Results Tables

- You create a results table in DESeq2 using the `results()` function.
- This table includes:
  - **log2 fold changes** (how much expression changes between groups, on a log scale)
  - **p values** (the statistical test result for whether there's a difference)
  - **adjusted p values** (p values corrected for multiple testing, **THIS** is the p-value we use here)
- If you don't give `results()` any extra arguments:
  - The comparison will be based on the **last variable in your design formula**. In our case, that's *dex*.
  - Because *dex* is a factor with levels `"untrt"` and `"trt"`, the default comparison will be:
    - the **last level** (`"trt"`)
    - **over the reference level** (`"untrt"`).

  - In other words, the log2 fold changes represent how gene expression changes in `"trt"` compared to `"untrt"`.

- Don't worry too much about the order of variables in your design formula.
  - You can always specify exactly which comparison you want by using the `name` or `contrast` arguments in the `results()` function. Here's what that could look like:

```
# Default behavior:
# Because dex is a factor with levels "untrt" (reference) and "trt" (non-reference),
# results(test) will compare trt vs untrt
res_default <- results(test)

# Making it explicit with contrast:
# Format: contrast = c("variable", "test_level", "reference_level")
res_contrast <- results(test, contrast = c("dex", "trt", "untrt"))
```

Because our ***untrt*** acts as a "control" for this dataset, it's fine to leave it as it, where we are looking at how gene expression changes in `"trt"` compared to `"untrt"`. That's why we can say things like "This gene was upregulated/downregulated in cells treated with dexamethasone", because ultimately it is a comparison to a control group, and we are interested in seeing changes in expression in the treated group.

Understanding DESeq2: Dispersion

Next, we are going to take a look at our dispersion plot from our DESeq2 results. This is essentially one of DESeq's "diagnostic" plots that shows the relationship between **gene expression strength** (average expression level of each gene) and the **natural variability across samples** (dispersion).

Let's take a look at our dispersion plot from our DESeq2 results:

```
plotDispEsts(test)
```

Axes

- **x-axis**: Mean of normalized counts
  - Think of this as **gene expression strength**, or how strongly a gene is expressed on average across all samples.
- **y-axis**: Dispersion estimate
  - This represents the **natural variability across samples** for each gene, after accounting for sequencing depth.

Points and Lines

- **Black dots**:
  - Each dot is a gene.
  - Its position shows the gene's mean expression (x) and raw dispersion (y).
- **Blue points** (if present):
  - Shrunk dispersion estimates.
  - These are the stabilized values DESeq2 actually uses in statistical testing.
  - Shrinkage is especially important for genes with low counts, where sample variability is noisy.
- **Red line**:
  - A fitted trend showing the typical relationship between expression strength and variability.
  - Genes with higher expression tend to show lower variability.

Why this plot is useful

- It's a **diagnostic tool** to check dispersion estimation.
- Most black dots should follow the red trend line.
- A good fit means your data are well-behaved for DESeq2's statistical model.
- If the dots are scattered far from the line, it may point to outliers or data quality issues.

In summary:
`plotDispEsts(test)` shows how **gene expression strength** relates to its **natural variability across samples**, and whether the DESeq2 model is fitting the data properly.

We can see here that our data pretty much follows the fitted trend, so we are good to continue!

- *Note*: most DE dispersion plots look something like this, so we are on the right track here!

Visualization of DEGs: Volcano Plots

A **volcano plot** is a simple way to visualize DESeq2 results. Essentially, data points form into a volcano shaped based on these parameters:

- **x-axis**: log2 fold change (effect size; how much is that gene differentially expressed)
- **y-axis**: –log10 adjusted p-value (significance)
- Genes that are both significant and strongly differentially expressed stand out in the top left and right corners.

To plot a volcano plot with `ggplot2`, we need to first make sure our data is ready to use. To do that, we are going to firtst make the `res` DESeq results object we made before into a dataframe. Then, we will subset our dataframe to make sure that we highlight our DEGs based on their p-value.

```
# Prepare a results dataframe for plotting
res_df <- as.data.frame(res)

# Add a new column 'significant' to the data frame 'res_df'
res_df_one <- res_df %>%
  mutate(
    # 'significant' will be "SIG" if the adjusted p-value is < 0.05
    # AND the absolute value of log2 fold change is >= 1

    # Otherwise, it will be "NS" (not significant)
    significant = if_else(
      padj < 0.05 & abs(log2FoldChange) >= 1,  # condition
      "SIG",                                   # value if TRUE
      "NS"                                     # value if FALSE
    )
  ) %>%

  # remove any NAs from df_res_one (usually cells without values)
  filter(!is.na(padj) & !is.na(log2FoldChange))

# Make the volcano plot with res_df
vol1 <- ggplot(res_df_one, aes(x = log2FoldChange, y = -log10(padj), color = significant)) +
  geom_point(alpha = 0.6, size = 1.5) +
  scale_color_manual(values = c("NS" = "grey", "SIG" = "red")) +
  theme_minimal() +
  labs(title = "Volcano Plot of DESeq2 Results",
       x = "log2 Fold Change",
       y = "-log10 Adjusted p-value")

vol1
```

Do you see the volcano plot? Great work!! We see that are significant genes, but in terms of logFC (magnitude of change) and p-values (significance value) are in red, while non-significant genes are in grey. **Fantastic work everyone!! You've just completed one of the most important steps in transcriptomic analyses!** Pat yourselves on the back!

We aren't done with our analysis yet though. Right now, our volcano plot highlights `SIG` genes in red—these are genes that show **both** a large log2 fold change and an adjusted p-value below 0.05. But what about genes with a very large log2 fold change whose adjusted p-value is **above** 0.05? Should we ignore them?

The answer is ***no***. A high log2 fold change indicates that the gene's expression differs substantially between our treatments (trt vs untrt), whether it is upregulated or downregulated. Even if they don't meet the strict statistical cutoff, these genes could still be biologically meaningful. For this reason, it's useful to define a few additional categories to capture these genes in our analysis.

Defining appropropriate categories We are going to add a categorical variable that encodes four possibilities for coloring our volcano plot:

1. Not significant at all (NS)
2. Only log2FoldChange > 1 (but not significant)
3. Only padj < 0.05 (but not high logFC)
4. Both padj < 0.05 and log2FoldChange >= 1

We are first going to use piping and dyplyr again to modify our original dataframe (`res_df`):

```
res_df_all <- res_df %>%
  mutate(
    # new column
    volcano_group = case_when(
      padj < 0.05 & abs(log2FoldChange) >= 1 ~ "Both",        # significant & large fold chang
      padj < 0.05 & abs(log2FoldChange) < 1 ~ "p-value < 0.05",  # significant only
      padj >= 0.05 & abs(log2FoldChange) >= 1 ~ "LogFC > 1",     # large fold change only
      TRUE ~ "NS"                                              # not significant
    )
  ) %>%

 # remove any NAs from df_res_all (usually cells without values)
  filter(!is.na(padj) & !is.na(log2FoldChange))
```

Let's stop here and just take a peak at the dataframe we just made:

```
head(res_df_all)
```

We can see we have a new column on our dataframe called *volcano_group*. This will be what `ggplot2` will use to plot all of our genes according to the variables I specified earlier. The cells in this group can be:

1. “Both” (p<0.05, logFC>1)
2. “p-value < 0.05” (sig only)
3. “LogFC > 1” (large FC only)
4. “NS” (not sig and small FC)

Now that we have the data where we want it, let’s go ahead and plot it.

For the plot, I want to add **cutoff lines** to visually indicate thresholds on my figure...

- **Horizontal line**: `-log10(0.05)` → marks the significance cutoff
- **Vertical lines**: `x = -1` and `x = 1` → mark the fold-change cutoff
- These lines help quickly identify which genes fall into which category

and **assign colors** to my genes based on category:

Category	Criteria	Color
NS	Not significant	Grey
LogFC > 1	High fold change only	Blue
p-value < 0.05	Significant only	Orange
Both	High fold change & significant	Red

```
# Create a volcano plot with colored categories
vol2 <- ggplot(res_df_all, aes(x = log2FoldChange, y = -log10(padj), color = volcano_group)) +

  # Add points for each gene
  geom_point(alpha = 0.8, size = 2) +
  # alpha = 0.8 makes points slightly transparent
  # size = 2 controls the point size

  # Manually assign colors to each category
  scale_color_manual(values = c(
    "NS" = "grey",
    "LogFC > 1" = "blue",
    "p-value < 0.05" = "orange",
    "Both" = "red"
  )) +

  # Add dashed horizontal line for p-value cutoff (0.05)
  geom_hline(yintercept = -log10(0.05), linetype = "dashed") +

  # Add dashed vertical lines for log2 fold-change cutoff (-1 and 1)
  geom_vline(xintercept = c(-1, 1), linetype = "dashed") +

  theme_minimal() +
```

```r
  # Label axes and legend
  labs(
    x = "log2 Fold Change",
    y = "-log10 Adjusted p-value",
    color = "Gene Category"
  )

# Display the plot
vol2
```

Interpreting the Volcano Plot Now, how does this volcano plot look compared to the old one we made earlier? It's the same plot, we just can visually see a lot more information about the individual genes.

This volcano plot visualizes **differentially expressed genes (DEGs)** in airway cells treated with dexamethasone compared to untreated control cells.

- Genes located **to the left** of the plot (negative log2 fold change) are **downregulated** in dexamethasone-treated cells relative to control.
- Genes located **to the right** of the plot (positive log2 fold change) are **upregulated** in dexamethasone-treated cells relative to control.

In other words, the x-axis indicates the **direction and magnitude of change** in gene expression, while the y-axis (-log10 adjusted p-value) represents **statistical significance**. Genes in the top corners of the plot (in red) are both highly differentially expressed and statistically significant.

Congratulations!

Great job! You've completed the DESeq2 tutorial and learned how to create a volcano plot. This is often the first step in differential gene expression analysis, but the results you've generated here can serve as the foundation for many other downstream analyses.

Mastering these steps is a key part of building your skills in **comprehensive RNA-seq analysis**. Be proud of what you've accomplished!